

# SOFTWARE REQUIREMENTS SPECIFICATION

*for*

## **SIM — Security Incident Mapping**

A Campus Security Incident Reporting Platform

Prepared for: SEN202 — Weekend 1 Development Exercise

Course Facilitator: Dr. O. O. Ajayi

Prepared by: [Your Full Name]

Matric Number: [Your Matric Number]

Version 1.0 — July 2026

## Table of Contents

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for the Security Incident Mapping (SIM) system. SIM is a web-based platform that allows students, staff, and landlords within a campus community to report security incidents — such as theft, assault, or other crimes — quickly and, where preferred, anonymously. This document is intended for the course facilitator, evaluators, and any developer who extends the prototype beyond this exercise.

## 1.2 Scope

SIM addresses the problem of underreported and slowly-escalated security incidents within campus communities, a contributing factor to broader community crime and insecurity as identified in the SEN202 Weekend 1 brief. The system in scope for this exercise is a mini web interface (prototype) that allows a user to submit an incident report with type, location, severity, and description, and view a live feed of reported incidents. Full production concerns — persistent multi-user databases, authentication hardening, geolocation mapping, and SMS/email alert delivery — are documented as intended functionality but are not required for the prototype, per the assignment's instruction to avoid unnecessary complexity.

## 1.3 Definitions, Acronyms and Abbreviations

- SIM — Security Incident Mapping (the system name)
- SRS — Software Requirements Specification
- UI — User Interface
- Admin — System Administrator responsible for user and platform management
- Security Personnel — Campus security staff who respond to and resolve reported incidents
- Anonymous Report — An incident report submitted without the reporter's identity attached

## 1.4 References

- SEN202 Weekend 1 Dev brief, issued 01/07/2026, Dr. O. O. Ajayi
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications (structure reference)

## 1.5 Overview

Section 2 gives an overall description of the product. Section 3 lists system features and functional requirements. Section 4 covers non-functional requirements. Section 5 describes external interface requirements. Section 6 explains the interaction of the system's major components.

# 2. Overall Description

## 2.1 Product Perspective

SIM is a new, standalone web-based system. It is not a replacement for official campus security or emergency services, but a supplementary reporting and awareness layer that sits between the community (students, staff, landlords) and campus security personnel.

## 2.2 Product Functions

- Allow community members to submit security incident reports (type, location, severity, description).
- Allow reports to be submitted anonymously.
- Display a live, chronological feed of reported incidents.
- Notify security personnel of new or high-severity reports.
- Allow security personnel to triage, respond to, and update the status of reports.

- Allow an administrator to manage users and view aggregate analytics.

## 2.3 User Classes and Characteristics

| ID                  | Requirement                                                                           | Priority         |
|---------------------|---------------------------------------------------------------------------------------|------------------|
| Student             | Primary reporter; occasional, on-the-go use, mostly via mobile browser.               | High             |
| Staff               | Reports incidents observed in the course of work; may also review campus-wide trends. | Medium           |
| Landlord / Resident | Reports incidents at off-campus lodges where students reside.                         | Medium           |
| Security Personnel  | Reviews, verifies, and responds to incoming reports.                                  | High             |
| System Admin        | Manages user accounts, permissions, and platform configuration.                       | Low (infrequent) |

## 2.4 Operating Environment

- Client: any modern web browser (Chrome, Edge, Firefox, Safari) on desktop or mobile.
- Server (future full build): standard web application server with HTTPS.
- Prototype scope: runs client-side as a static HTML/CSS/JavaScript interface, requiring no server installation.

## 2.5 Design and Implementation Constraints

- Submission timeline required a lightweight, low-complexity implementation for the prototype phase.
- No persistent backend/database is implemented in the prototype; incident data exists only for the active browser session.
- The system must remain simple enough for non-technical community members to use without training.

## 2.6 Assumptions and Dependencies

- Users have access to an internet-connected device with a web browser.
- A future production version will require a backend service and database (e.g. Node.js/Express with MongoDB or a similar stack) plus hosting.
- Campus security personnel are assumed to be willing participants who monitor the platform.

# 3. System Features (Functional Requirements)

## 3.1 Incident Reporting

Allows a user to submit a new security incident report.

| ID     | Requirement                                                                                          | Priority |
|--------|------------------------------------------------------------------------------------------------------|----------|
| FR-1.1 | The system shall allow a user to select an incident type (e.g. theft, assault, suspicious activity). | High     |
| FR-1.2 | The system shall allow a user to enter a free-text location for the incident.                        | High     |
| FR-1.3 | The system shall allow a user to set a severity level (Low, Medium, High).                           | High     |
| FR-1.4 | The system shall allow a user to enter a free-text description of the incident.                      | High     |
| FR-1.5 | The system shall timestamp every report at the time of submission.                                   | Medium   |

## 3.2 Anonymous Reporting

| ID     | Requirement                                                                                            | Priority |
|--------|--------------------------------------------------------------------------------------------------------|----------|
| FR-2.1 | The system shall allow a user to toggle a report as anonymous before submission.                       | High     |
| FR-2.2 | When anonymous, the system shall not display or store the reporter's role/identity against the report. | High     |

### 3.3 Incident Feed / Map View

| ID              | Requirement                                                                       | Priority |
|-----------------|-----------------------------------------------------------------------------------|----------|
| FR-3.1          | The system shall display submitted reports in a live, reverse-chronological feed. | High     |
| FR-3.2          | The system shall visually distinguish reports by severity level.                  | Medium   |
| FR-3.3 (future) | The system shall plot incident locations on an interactive campus map.            | Low      |

### 3.4 Notification / Alerts

| ID              | Requirement                                                                | Priority |
|-----------------|----------------------------------------------------------------------------|----------|
| FR-4.1 (future) | The system shall notify security personnel when a new report is submitted. | High     |
| FR-4.2 (future) | The system shall escalate notification urgency for High-severity reports.  | Medium   |

### 3.5 Verification, Response & Status Update

| ID              | Requirement                                                                                               | Priority |
|-----------------|-----------------------------------------------------------------------------------------------------------|----------|
| FR-5.1 (future) | The system shall allow security personnel to mark a report as verified.                                   | High     |
| FR-5.2 (future) | The system shall allow security personnel to update a report's status (e.g. Open, In Progress, Resolved). | High     |

### 3.6 Administration & Analytics

| ID              | Requirement                                                                                     | Priority |
|-----------------|-------------------------------------------------------------------------------------------------|----------|
| FR-6.1 (future) | The system shall allow an admin to manage user accounts and roles.                              | Medium   |
| FR-6.2 (future) | The system shall provide an aggregate dashboard of incident trends by type, location, and time. | Low      |

Note: items marked “(future)” extend beyond the current mini web interface prototype but are documented here to give a complete picture of the intended system, as required by this SRS.

## 4. Non-Functional Requirements

### 4.1 Performance

- The report submission form shall respond and update the feed within 1 second on the client side.

### 4.2 Security

- Anonymous reports shall not retain any identifying metadata.
- A production version shall transmit all data over HTTPS and store sensitive data encrypted at rest.

### 4.3 Usability

- The interface shall be usable by a first-time visitor without instructions, given its role in urgent situations.
- The interface shall be responsive and usable on both desktop and mobile screen sizes.

### 4.4 Reliability / Availability

- A production version of the system shall target 99% uptime, given its safety-related purpose.

## 4.5 Scalability

- The system architecture shall allow the incident feed to scale from a single-session prototype to a multi-user, persistent, campus-wide deployment.

## 4.6 Maintainability

- The codebase shall separate presentation (UI), logic, and data concerns to simplify future extension (e.g. adding a real backend).

# 5. External Interface Requirements

## 5.1 User Interfaces

A single-page web interface comprising a report submission form and a live incident feed, as delivered in the accompanying mini web interface prototype.

## 5.2 Hardware Interfaces

None required beyond a standard internet-connected computer, tablet, or smartphone.

## 5.3 Software Interfaces

Prototype: none (self-contained HTML/CSS/JavaScript). Future production version: REST API between client and server, and a database interface (e.g. MongoDB/PostgreSQL).

## 5.4 Communication Interfaces

Prototype: none. Future production version: HTTPS for all client-server communication; optional SMS/email gateway integration for alerting.

# 6. System Architecture and Component Interaction

The diagram and flow below explain how the major components of SIM interact, satisfying the requirement to clearly present component interaction alongside functional and non-functional requirements.

## 6.1 Major Components

- Presentation Layer (Web UI): the report form and incident feed the user directly interacts with.
- Application Logic Layer: validates input, applies severity/anonymity rules, and manages the in-session report list (prototype) or business logic (production).
- Data Layer: in the prototype, an in-memory JavaScript array holding submitted reports for the active session; in production, a persistent database.
- Notification Component (future): pushes alerts to security personnel when new or high-severity reports arrive.
- Admin/Analytics Component (future): aggregates stored reports into dashboards and manages user accounts.

## 6.2 Interaction Flow

1. A user (student, staff, or landlord) opens the Web UI and completes the incident report form.
2. On submission, the Application Logic Layer validates the input and timestamps the report.
3. The validated report is passed to the Data Layer for storage (session memory in the prototype; database in production).
4. The Presentation Layer re-renders the incident feed to reflect the newly stored report.
5. (Production) The Notification Component detects the new report and alerts Security Personnel.

- 6. (Production) Security Personnel interact with the Application Logic Layer to verify the report and update its status, which is written back to the Data Layer and reflected in the feed.
- 7. (Production) The Admin/Analytics Component periodically reads from the Data Layer to generate trend dashboards for the System Admin.

This flow corresponds to the actors and use cases documented in the accompanying Use-Case Diagram, and the root-cause categories documented in the accompanying Ishikawa (fishbone) diagram, which together informed the functional requirements in Section 3.